



2026-1 텍스트마이닝

과제_3 Word2Vec_구현

세종대학교
소프트웨어 학과
학번: 22011831
이름: 김형규



Step 1

```
corpus = [
    "he is a king",
    "she is a queen",
    "he is a man",
    "she is a woman",
    "seoul is korea capital",
    "tokyo is japan capital",
    "paris is france capital"
]

# 토큰화 및 단어 중복 제거
words = " ".join(corpus).split()
word_list = list(set(words)) # 🐵 단어 중복을 제거하는 파이썬 내장 함수를 사용하세요.

word_dict = {w: i for i, w in enumerate(word_list)}
number_dict = {i: w for i, w in enumerate(word_list)}
vocab_size = len(word_dict)

print("단어 사전 크기:", vocab_size)
print("단어-인덱스 매핑:", word_dict)
```

코드 작성 이유

1. 코퍼스를 토큰화하고 set()으로 중복을 제거해 word_dict / number_dict를 만들었다.
2. 이후 단계의 원-핫 벡터 차원과 출력층 크기가 모두 vocab_size에 의존하므로, 사전 구축이 가장 먼저 수행되어야 한다.

분석 결과

1. 총 어휘 수는 15개(vocab_size=15)로 매우 작은 toy 코퍼스이다. 작은 어휘는 학습 시간을 짧게 해주지만, 의미 관계가 형성되려면 동일 패턴이 여러 번 등장해야 한다.
2. 코퍼스 내에 'king/queen', 'man/woman', 'seoul/tokyo/paris' 같은 의미적 짝이 의도적으로 배치되어 있어, Skip-gram이 잘 학습되면 시각화에서 군집이 형성될 것이라 기대된다.

단어 사전 크기: 15

단어-인덱스 매핑: {'seoul': 0, 'she': 1, 'is': 2, 'king': 3, 'queen': 4, 'france': 5, 'a': 6, 'japan': 7, 'paris': 8, 'korea': 9,



Step 2

```
window_size = 1
skip_grams = []

for sentence in corpus:
    tokens = sentence.split()
    for i in range(len(tokens)):
        target = word_dict[tokens[i]]
        # 윈도우 사이즈 내의 주변 단어 찾기
        for j in range(max(0, i - window_size), min(len(tokens), i + window_size + 1)):
            if i != j:
                context = word_dict[tokens[j]] # 🐵 tokens 리스트에서 주변 단어의 인덱스(j)를 참조해 사전에 넣으세요.
                skip_grams.append([target, context])

print(f"생성된 학습 데이터 개수: {len(skip_grams)}")
print("첫 5개 데이터 (Target, Context):", skip_grams[:5])

✓ 0.0s
```

코드 작성 이유

1. Skip-gram은 중심 단어로 주변 단어를 예측하므로 (target, context) 쌍 자체가 학습 데이터가 된다. window_size 내의 인덱스를 순회해 자기 자신을 제외한 모든 주변 단어를 짝지어 주었다.
2. tokens[j]를 word_dict로 매핑해 인덱스 형태로 저장한 이유는 CrossEntropyLoss가 정답 라벨로 정수 인덱스를 요구하기 때문이다.

분석 결과

1. window_size=1이라는 좁은 윈도우에서도 7개 문장만으로 42개의 학습 쌍이 생성됐다. 같은 문장 안에서 (target, context)와 (context, target)이 모두 들어가 양방향 학습이 자연스럽게 이뤄진다.
2. 윈도우가 작으면 직접 인접 단어만 학습하므로 의미 군집은 명확해지지만, 멀리 떨어진 단어 간 관계는 학습되지 않는다.

생성된 학습 데이터 개수: 42

첫 5개 데이터 (Target, Context): [[14, 2], [2, 14], [2, 6], [6, 2], [6, 3]]



Step 3

```
class Word2Vec(nn.Module):
    def __init__(self, vocab_size, embedding_size):
        super(Word2Vec, self).__init__()
        # V -> D (Projection / Embedding)
        self.W = nn.Linear(vocab_size, embedding_size, bias=False) # 👉 vocab_size와 embedding_size를 입력하세요.
        # D -> V (Output)
        self.WT = nn.Linear(embedding_size, vocab_size, bias=False) # 👉 embedding_size와 vocab_size를 입력하세요.

    def forward(self, X):
        hidden_layer = self.W(X)
        output_layer = self.WT(hidden_layer)
        return output_layer

# 2차원 그래프에 시각화하기 위해 임베딩 사이즈는 2로 설정합니다.
embedding_size = 2
model = Word2Vec(vocab_size, embedding_size)
```

코드 작성 이유

1. 입력층에서 은닉층으로 가는 가중치 행렬W와 은닉층에서 출력층으로 가는 행렬WT를 정의한다.
2. 은닉층의 크기(embedding_size)가 바로 우리가 얻고자 하는 단어 벡터의 차원수가 된다. 시각화를 위해 초기 설정은 2차원으로 지정하였다.

분석 결과

1. Word2Vec에서는 은닉층에 활성화 함수가 없다. 은닉층에 비선형 활성화 함수(예: ReLU)를 넣으면 단어 벡터들이 복잡하게 꺾이게 되어, 단어 사이의 직관적인 선형 관계가 깨질 확률이 높다.
2. 최종 임베딩으로 사용하는 것은 W의 가중치 행렬이며, WT는 학습 보조용이다. 모델 파라미터 수는 $V \cdot D + D \cdot V = 15 \cdot 2 + 2 \cdot 15 = 60$ 개로 매우 작다.



Step 4

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

def get_one_hot(word_idx, vocab_size):
    x = torch.zeros(vocab_size)
    x[word_idx] = 1.0 # 🖱 해당 word_idx 위치를 1.0으로 설정하여 원-핫 벡터를 완성하세요.
    return x

# 원-핫 인코딩 결과 중간 확인
print("One-hot vector for index 0:", get_one_hot(0, vocab_size))
```

코드 작성 이유

1. 출력은 어휘 전체에 대한 점수이므로 다중 클래스 분류로 보고 CrossEntropyLoss를 선택했다. 이 손실은 내부에 softmax가 포함되어 별도 softmax 호출이 필요 없다.
2. get_one_hot()은 단어 인덱스를 입력받아 V차원 원-핫 벡터로 변환하여 입력 형태를 맞춰주는 함수이다.

분석 결과

1. 원-핫 벡터를 W에 통과시키는 연산은 사실상 W의 해당 단어 행을 그대로 가져오는 'lookup'과 동일하다. 행렬곱처럼 보이지만 실제로는 임베딩 테이블 조회이다.
2. 출력에서 인덱스 0 위치만 1.0인 길이 15짜리 벡터가 잘 만들어졌음을 확인했다.

```
One-hot vector for index 0: tensor([1., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```



Step 5

```
epochs = 5000
losses = []

for epoch in range(epochs):
    loss_val = 0
    for target, context in skip_grams:
        # 중심 단어의 원핫 벡터 생성
        x = get_one_hot(target, vocab_size)
        # 주변 단어의 실제 정답 인덱스
        y_true = torch.tensor([context], dtype=torch.long)

        optimizer.zero_grad()
        y_pred = model(x)

        loss = criterion(y_pred.unsqueeze(0), y_true)
        loss.backward() # 🐱 역전파 수행 (backward)
        optimizer.step() # 🐱 가중치 업데이트 (step)

    loss_val += loss.item()

avg_loss = loss_val / len(skip_grams)
losses.append(avg_loss)
if epoch % 1000 == 0:
    print(f'Epoch: {epoch}, Loss: {avg_loss:.4f}')
```

코드 작성 이유

1. 각 step마다 중심단어의 원핫벡터(x), 실제 정답(y_true), X에 대한 모델의 예측(y_pred)를 생성하여 모델을 훈련한다. Loss를 줄이는 방향으로 역전파를 통해 가중치를 미세 조정한다.
2. epoch마다 평균 loss를 기록해 학습 진행 상황을 확인하고 시각화하기 쉽도록 했다.

분석 결과

1. 초기 loss는 약 2.71에서 시작해 4000 epoch 동안 약 1.14 까지 지속적으로 감소하였다. 모델이 toy 코퍼스의 가능한 (target, context) 분포를 거의 다 학습했음을 의미한다.
2. loss가 0이 아니라 1.14 근처에서 멈추는 이유는, 같은 target이 다수의 다른 context를 가지므로(예: 'is' → he/she/a/korea/japan...) 한 번의 예측으로 모든 정답을 동시에 맞출 수 없기 때문이다.

```
Epoch: 0, Loss: 2.7158
Epoch: 1000, Loss: 1.1441
Epoch: 2000, Loss: 1.1426
Epoch: 3000, Loss: 1.1418
Epoch: 4000, Loss: 1.1412
```



Step 5

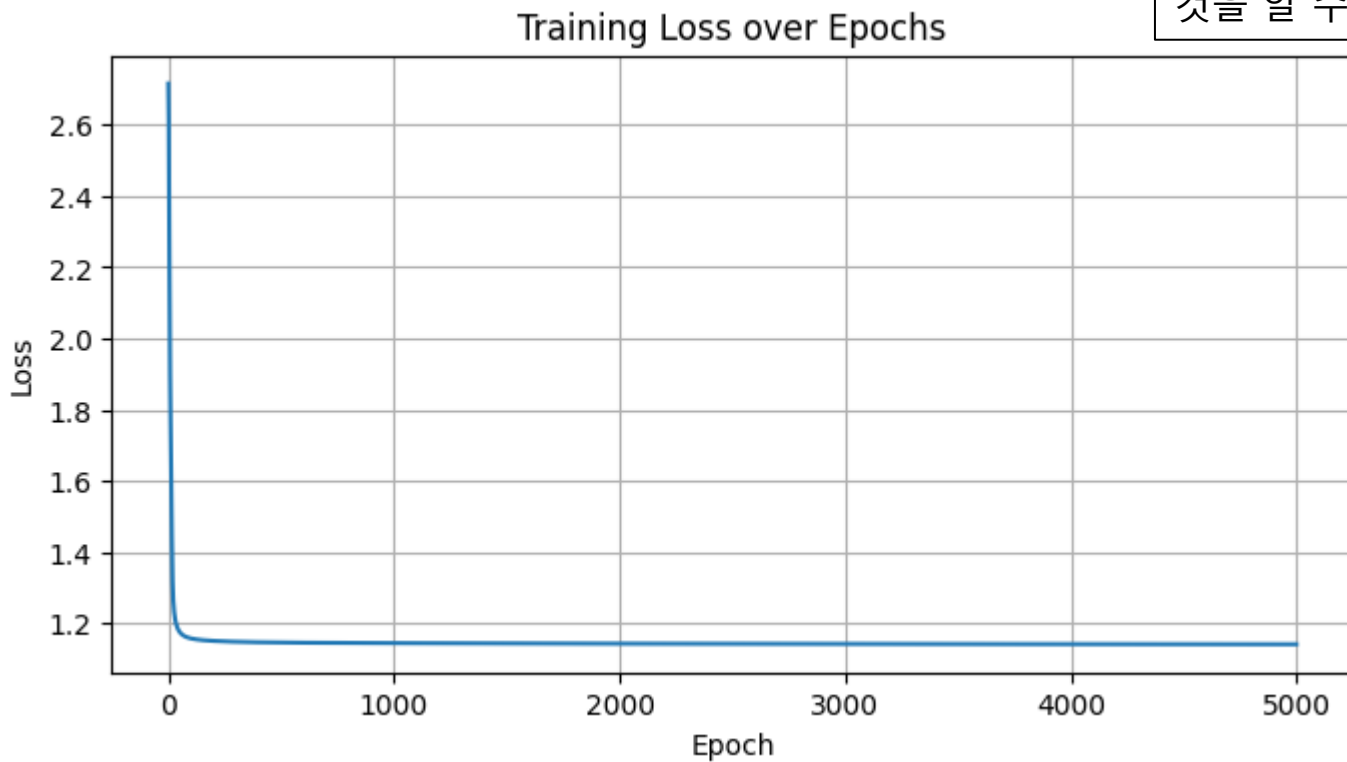
```
plt.figure(figsize=(8, 4))
plt.plot(losses)
plt.title("Training Loss over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()
```

코드 작성 이유

모델 학습시 저장해놓았던 Loss를 통해 에포크에 따른 Loss를 시각화하여 그래프로 그려준다.

분석 결과

훈련 초기에 Loss값이 급격히 떨어지고, 그 이후로는 서서히 줄어드는 것을 알 수 있다.



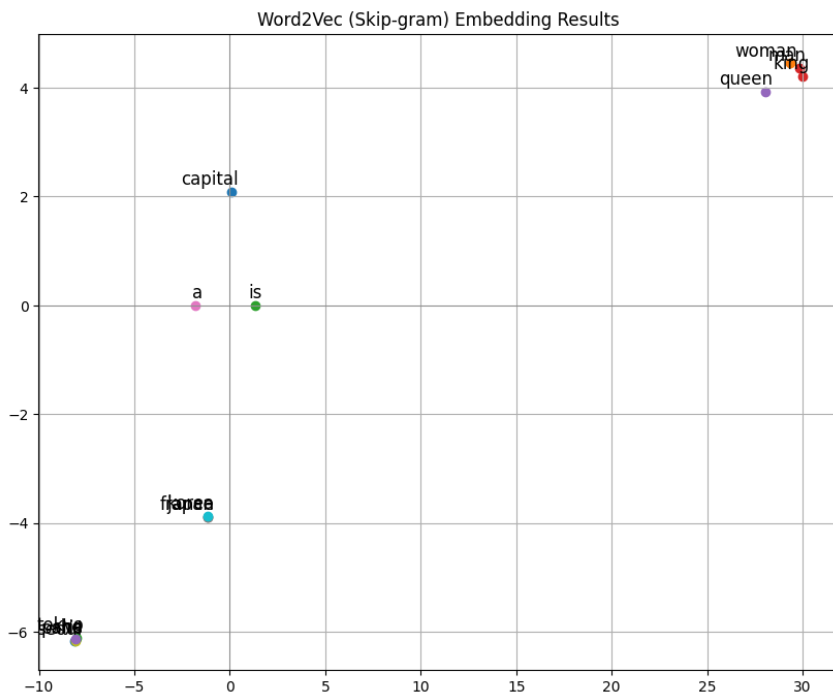


Step 6

```
# 가중치 텐서를 numpy 배열로 변환
W_embed = model.W.weight.data.numpy().T

plt.figure(figsize=(10, 8))
for i, word in enumerate(word_list):
    x, y = W_embed[i][0], W_embed[i][1]
    plt.scatter(x, y)
    plt.annotate(word, (x, y), xytext=(5, 2), textcoords='offset points', ha='right', va='bottom', fontsize=12)

plt.title("Word2Vec (Skip-gram) Embedding Results")
plt.axhline(0, color='grey', lw=0.5)
plt.axvline(0, color='grey', lw=0.5)
plt.grid(True)
plt.show()
```



코드 작성 이유

1. 학습된 W의 행 벡터가 곧 단어의 임베딩이므로 가중치 행렬 W를 numpy 배열로 변환했다. PyTorch의 nn.Linear는 내부적으로 가중치 행렬을 (D, V) 순서로 저장하기 때문에 .T를 취해 (V, D) 형태로 만든 뒤 단어별 좌표로 사용했다.
2. embedding_size=2이므로 별도 차원 축소(PCA, t-SNE) 없이 그대로 산점도로 시각화 가능하다.

분석 결과

1. 결과 그래프에서 의미적으로 비슷한 단어가 같은 영역에 군집을 이루었다. 우측 상단에 king/queen/man/woman 같은 사람·성별 관련 단어가 모였고, 좌측 하단에 도시명(seoul/Tokyo/paris)과 대명사(he/she)가, 중심에 더 가까운 쪽에는 국가명(korea/Japan/France)이 모여 있다.
2. 조사·관사 역할의 'is', 'a'는 원점 근처에 자리잡고 있는데, 거의 모든 문장에 등장해 특정 카테고리에 치우치지 않은 결과로 해석된다.
3. 2차원 평면상에서 의미가 비슷한 단어들이 서로 가깝게 배치되거나 일정한 방향성을 갖는 경향을 보인다는 것을 알 수 있다.



Step 7

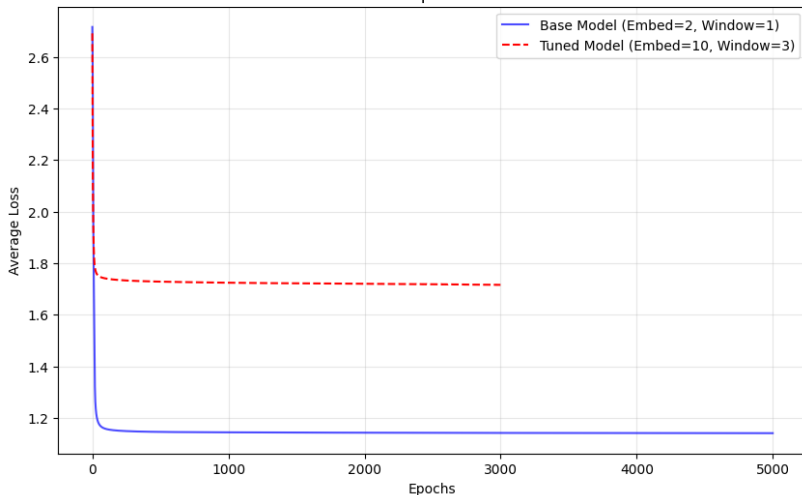
```
# 하이퍼파라미터 설정 변경
tuning_embedding_size = 10
tuning_window_size = 3
tuning_epochs = 3000

# 데이터 재구성 (window_size 변경)
tuning_skip_gram(variable) corpus: list[str]
for sentence in corpus:
    tokens = sentence.split()
    for i in range(len(tokens)):
        target = word_dict[tokens[i]]
        for j in range(max(0, i - tuning_window_size), min(len(tokens), i + tuning_window_size + 1)):
            if i != j:
                context = word_dict[tokens[j]]
                tuning_skip_grams.append([target, context])

# 모델 및 옵티마이저 초기화
tuning_model = Word2Vec(vocab_size, tuning_embedding_size)
tuning_criterion = nn.CrossEntropyLoss()
tuning_optimizer = optim.Adam(tuning_model.parameters(), lr=0.01)
tuning_losses = []

print(f"--- 새 모델 학습 시작 (Embed: {tuning_embedding_size}, Window: {tuning_window_size}) ---")
```

Word2Vec Loss Comparison: Base vs Tuned



코드 작성 이유

1. embedding_size를 10으로 늘려 표현력을 확장하고, window_size를 3으로 늘려 더 넓은 문맥을 학습 신호로 사용했다. 두 변화가 학습 양상에 미치는 영향을 함께 관찰하려는 의도이다.
2. Epoch는 5000→3000으로 줄여 학습 비용 변화도 같이 점검했다.

분석 결과

1. window_size=3로 확장하니 학습 쌍의 개수가 늘었다. 한 target 단어가 예측해야 하는 context의 종류도 많아졌고, 맞춰야 할 정답이 많아지면 각 정답에 할당되는 확률이 분산된다. 결과적으로 평균 Loss는 Base Model의 오차(약 1.14)보다 상승하였다.
2. Loss 값이 더 크다고 해서 모델 품질이 나쁜 것은 아니다. Embedding_size의 증가로 의미 표현 공간이 더 풍부해져 단어 간 미세한 차이까지 표현할 수 있게 되었다. window_size의 증가로 더 많은 단어 사이의 문맥을 학습할 수 있게 되었다.



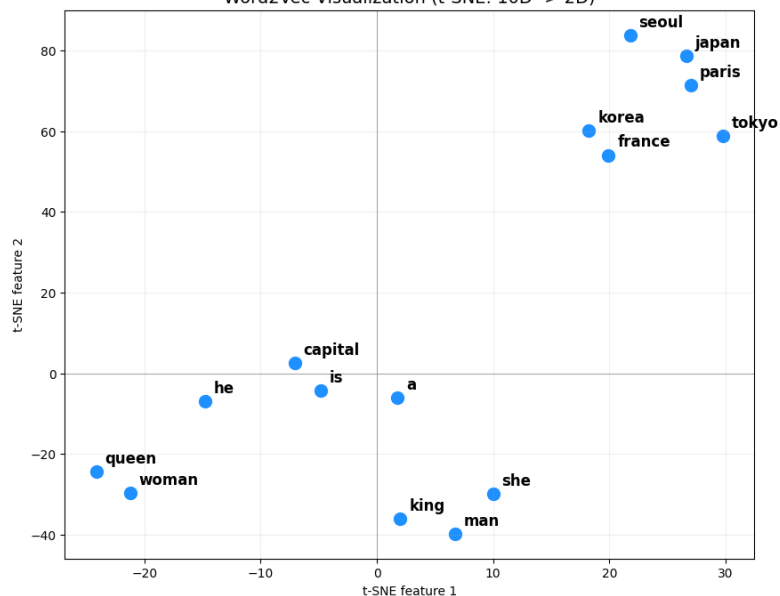
Step 7-1

```
# --- 1. 학습된 10차원 임베딩 가중치 추출 ---
W_10d = tuning_model.W.weight.data.numpy().T

# --- 2. t-SNE를 이용한 차원 축소 (10D -> 2D) ---
# n_samples가 15개이므로 perplexity는 보통 (n_samples - 1) / 3 보다 작아야 합니다.
# 여기서는 아주 작은 값인 3~5 정도로 설정합니다.
tsne = TSNE(n_components=2, perplexity=3, random_state=42, learning_rate='auto', init='pca')
W_tsne = tsne.fit_transform(W_10d)
```

```
# --- 3. 시각화 ---
plt.figure(figsize=(10, 8))

for i, word in enumerate(word_list):
    x, y = W_tsne[i][0], W_tsne[i][1]
    plt.scatter(x, y, color='dodgerblue', s=100)
    # 단어 라벨 표시
    plt.annotate(word, (x, y), xytext=(7, 7), textcoords='offset points',
                 fontsize=12, fontweight='bold')
```



코드 작성 이유

1. t-SNE는 데이터 간의 '국소적 유사성(Local Structure)'을 보존하는 데 최적화되어 있다. 즉, 10차원 공간에서 조금이라도 가까웠던 단어들을 2차원 평면에서 억지로라도 강하게 뭉쳐서(Cluster) 보여주는 특징이 있다.
2. Perplexity는 한 점이 주변 몇 개의 이웃을 고려할지 결정하는 파라미터로, 현재 규모가 작은 Toy코퍼스 이므로 작게(3) 설정하였다.

분석 결과

1. seoul, Japan, korea 등 나라와 관련된 단어들이 많이 뭉쳐있고, queen, woman과 같은 단어들이 많이 뭉쳐있다.
2. 하지만 king-queen, man-woman이 조금 떨어져있다는 것은 모델이 고차원 개념을 충분히 학습하지 못했다는 것을 의미하고, 이것은 toy 코퍼스의 데이터 양이 적기 때문인 것으로 추정된다.
3. 따라서 모델의 성능을 높이기 위해서는 모델의 구조를 키우는 것보다 더 많은 양질의 데이터가 필요하다는 것을 보여준다.